

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

КУРСОВАЯ РАБОТА

«Разработка Android-приложения “Мессенджер” с использованием архитектуры MVVM и фоновой синхронизации данных»

Выполнила:

Бахарева Мария Александровна, К3442

Преподаватель:

Шатинский Григорий Сергеевич

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Архитектура приложения и навигация	4
2 Управление состоянием и MVVM	7
3 Работа с данными и офлайн-доступ	10
4 Улучшение интерфейса и фоновая синхронизация	13
ЗАКЛЮЧЕНИЕ	16

ВВЕДЕНИЕ

В настоящее время мобильные приложения являются неотъемлемой частью повседневной жизни пользователей, а платформа Android занимает лидирующие позиции на рынке мобильных операционных систем. Разработка Android-приложений требует не только реализации пользовательского интерфейса, но и построения устойчивой архитектуры, корректной работы с данными, обработки системных событий и поддержки фоновых процессов. В связи с этим изучение принципов построения современных Android-приложений является актуальной задачей в рамках подготовки специалистов в области разработки программного обеспечения.

Целью данной курсовой работы является разработка Android-приложения «Мессенджер», реализующего базовый функционал обмена сообщениями с использованием архитектуры MVVM, сетевого взаимодействия, локального хранения данных и фоновой синхронизации. Приложение разрабатывалось поэтапно в рамках выполнения серии лабораторных работ, каждая из которых расширяла функциональность и архитектурную сложность проекта.

В рамках курсовой работы были поставлены следующие задачи: разработать пользовательский интерфейс приложения с использованием Activity, Fragment и встроенной навигации; реализовать управление состоянием приложения с применением ViewModel; организовать загрузку данных из сети и их сохранение в локальную базу данных для обеспечения офлайн-доступа; улучшить пользовательский интерфейс с применением компонентов Material Design; настроить фоновую синхронизацию данных и систему уведомлений, а также обеспечить корректную реакцию приложения на изменения сетевого состояния и системные события.

Разработанное приложение включает несколько экранов, доступных через нижнюю панель навигации, обеспечивает сохранение пользовательских данных при изменении конфигурации устройства, поддерживает работу без подключения к сети и выполняет периодическое обновление данных в фоновом режиме. Все компоненты приложения реализованы с соблюдением принципов разделения ответственности и расширяемости, что обеспечивает удобство дальнейшего развития и сопровождения проекта.

1 Архитектура приложения и навигация

В рамках курсовой работы было разработано Android-приложение «Мессенджер», предназначенное для отображения ленты сообщений, управления данными профиля пользователя и настройки параметров приложения. Реализация велась поэтапно на основе четырех лабораторных работ, каждая из которых последовательно расширяла функциональность и архитектурную сложность проекта, формируя целостное приложение.

На начальном этапе была создана базовая структура приложения с одной главной активностью, выступающей точкой входа. Внутри активности реализована навигация между основными экранами приложения с использованием нижней панели навигации. Для переходов между экранами применялись стандартные инструменты Android Navigation - NavController и Navigation Graph (рисунок 1), что позволило централизованно описать маршруты и упростить управление фрагментами. В приложении были реализованы три основных экрана: лента сообщений, профиль пользователя и настройки. Для контроля корректности работы компонентов в процессе разработки было добавлено логирование основных этапов жизненного цикла Activity и Fragment, что позволило отслеживать их создание, запуск и завершение.



```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)
    requestPermissionsIfNeeded()

    Log.i(TAG, msg = "onCreate")

    val navHost =
        supportFragmentManager.findFragmentById( id = R.id.nav_host_fragment) as NavHostFragment
    val navController = navHost.navController
    binding.bottomNav.setupWithNavController(navController)
}
```

Рисунок 1 - MainActivity с настройкой BottomNavigation и NavController (подключение Navigation Graph)

На следующем этапе была внедрена архитектура MVVM, обеспечивающая разделение ответственности между пользовательским интерфейсом и логикой приложения. Для экранов профиля и настроек были созданы ViewModel, предназначенные для хранения и управления состоянием пользовательских данных. Экран профиля получил возможность редактирования имени и статуса пользователя, которые сохранялись во ViewModel и автоматически отображались в интерфейсе. Экран настроек

включал переключатель темы, состояние которого также хранилось во ViewModel. Для реактивного обновления интерфейса использовалась LiveData, благодаря чему изменения состояния немедленно отражались в UI. Такой подход обеспечил сохранность данных при повороте экрана и других изменениях конфигурации устройства.

Дальнейшее развитие приложения было связано с реализацией работы с данными. Для загрузки сообщений был использован внешний API, доступ к которому осуществлялся через Retrofit с применением корутин. Для обеспечения офлайн-доступа была реализована локальная база данных Room, в которой сохранялись полученные сообщения. Взаимодействие между сетевым источником данных и локальным хранилищем было инкапсулировано в репозитории, который выступает единой точкой доступа к данным для ViewModel. При наличии сети репозиторий загружает данные из API и сохраняет их в базу, а при отсутствии сети - предоставляет данные из локального хранилища. Отображение списка сообщений на экране ленты было реализовано с использованием RecyclerView.

```
14 class MessageRepository(  
15     private val api: MessageApi,  
16     private val dao: MessageDao  
17 ) {  
18  
19     1 Usage  
20     val messagesFlow: Flow<List<Message>> =  
21         dao.getMessagesFlow().map { list ->  
22             list.map { it.toDomain() }  
23         }  
24  
25     2 Usages  
26     suspend fun refreshMessages(): Result<Unit> {  
27         return try {  
28             val likedMap = dao.getLikedMap().associate { it.id to it.liked }  
29  
30             val dto = api.getMessages()  
31             val entities = dto.map { it.toEntity() }.map { e ->  
32                 e.copy(liked = likedMap[e.id] ?: false)  
33             }  
34  
35             dao.clearAll()  
36             dao.insertAll( messages = entities)  
37  
38             Result.success( value = Unit)  
39         } catch (e: Exception) {  
40             Result.failure( exception = e)  
41         }  
42     }  
43 }
```

Рисунок 2 - MessageRepository, демонстрирующий логику получения данных из сети и базы Room

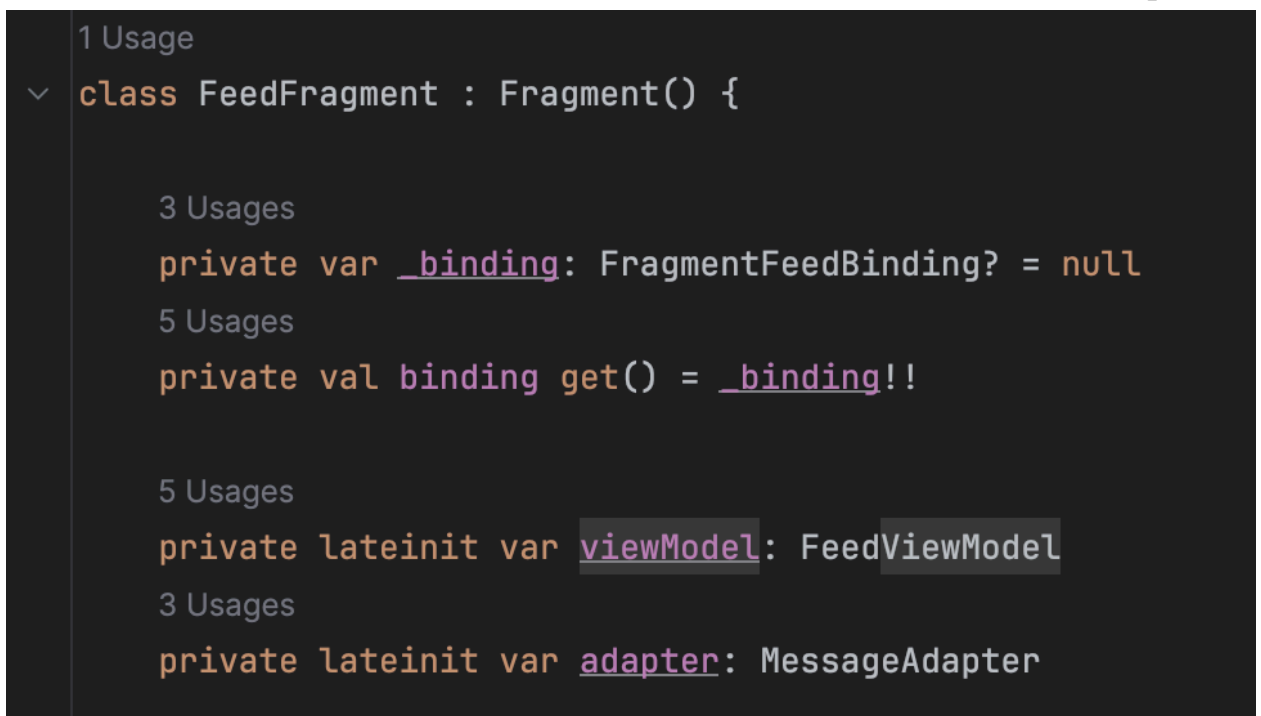
На завершающем этапе были улучшены пользовательский интерфейс и пользовательские сценарии. Список сообщений был дополнен кастомным отображением элементов, включающим аватар, имя отправителя и текст сообщения, а также возможностью «лайкать» сообщения с изменением иконки. Для приведения интерфейса к современным рекомендациям были использованы компоненты Material Design. Кроме того, была настроена фоновая синхронизация сообщений с помощью WorkManager, который периодически обновляет данные при наличии сети. При успешной синхронизации приложение отображает системное уведомление о получении новых данных. Также была реализована обработка разрешений, необходимых для корректной работы приложения, и обеспечена стабильная работа при изменении сетевого состояния.

Таким образом, в рамках выполнения лабораторных работ было создано Android-приложение, объединяющее навигацию, управление состоянием, работу с сетью и локальным хранилищем данных, а также механизмы фоновой синхронизации и уведомлений. Полученная структура приложения соответствует современным подходам к разработке Android-приложений и может служить основой для дальнейшего расширения функциональности.

2 Управление состоянием и MVVM

Архитектура разрабатываемого приложения построена на основе паттерна MVVM (Model–View–ViewModel), который обеспечивает четкое разделение ответственности между слоями пользовательского интерфейса, бизнес-логики и работы с данными. Такой подход позволил повысить устойчивость приложения к изменениям конфигурации, упростить тестирование и обеспечить масштабируемость проекта при дальнейшем расширении функциональности.

Слой View представлен Activity и Fragment, которые отвечают исключительно за отображение данных и обработку пользовательских действий. View не содержит бизнес-логики и не выполняет операций с данными напрямую, а взаимодействует с ViewModel, подписываясь на изменения состояния. В рамках приложения основная активность используется как контейнер для фрагментов и навигации, а каждый экран реализован в виде отдельного Fragment (рисунок 3). Такой подход соответствует рекомендациям Android и позволяет гибко управлять жизненным циклом экранов.



```
1 Usage
▼ class FeedFragment : Fragment() {

    3 Usages
    private var _binding: FragmentFeedBinding? = null

    5 Usages
    private val binding get() = _binding!!

    5 Usages
    private lateinit var viewModel: FeedViewModel

    3 Usages
    private lateinit var adapter: MessageAdapter
```

Рисунок 3 - FeedFragment с подпиской на состояние ViewModel и обновлением UI через Flow или LiveData

Слой ViewModel отвечает за хранение и обработку состояния экранов. ViewModel не зависит от жизненного цикла View и сохраняет данные при повороте экрана и других изменениях конфигурации. В приложении

ViewModel используется для управления состоянием профиля пользователя, настроек приложения и ленты сообщений. Для асинхронной работы и реактивного обновления интерфейса применяются корутины и StateFlow, что позволяет эффективно обрабатывать потоки данных и изменения состояния. В ViewModel также реализовано логирование этапов жизненного цикла, что упрощает отладку и анализ поведения приложения.

Слой Model включает в себя источники данных и бизнес-логику приложения. Для централизованного доступа к данным был реализован репозиторий, который инкапсулирует работу с сетевым API и локальной базой данных. Репозиторий определяет, из какого источника следует получать данные в зависимости от текущего состояния сети, и предоставляет ViewModel (рисунок 4) единый интерфейс для взаимодействия с данными. Такой подход исключает дублирование логики и снижает связанность между компонентами приложения.

```
4 Usages
class FeedViewModel(
    application: Application,
    private val repository: MessageRepository
) : AndroidViewModel(application) {

    22 Usages
    private val _state = MutableStateFlow( value = FeedUiState())
    1 Usage
    val state: StateFlow<FeedUiState> = _state

    init {
        Log.i( tag = "FeedViewModel", msg = "init")
        observeMessages()
        observeNetwork()
        refresh()
    }
}
```

Рисунок 4 - FeedViewModel, демонстрирующий работу с репозиторием и обновление состояния UI

Для хранения данных в офлайн-режиме используется база данных Room, включающая Entity, DAO и Database. Доступ к данным осуществляется через Flow, что позволяет автоматически обновлять интерфейс при изменении содержимого базы данных. Сетевое

взаимодействие реализовано с помощью Retrofit и корутин, что обеспечивает неблокирующую загрузку данных и корректную обработку ошибок. Таким образом, приложение корректно работает как при наличии подключения к сети, так и при его отсутствии.

Взаимодействие между слоями приложения организовано таким образом, что изменения данных автоматически отражаются в пользовательском интерфейсе без необходимости ручного обновления. ViewModel выступает связующим звеном между UI и источниками данных, обеспечивая единый поток данных и предсказуемое поведение приложения. Использование архитектуры MVVM в совокупности с современными инструментами Android позволило создать устойчивую и расширяемую архитектуру, соответствующую актуальным рекомендациям платформы.

3 Работа с данными и офлайн-доступ

В рамках третьей лабораторной работы функциональность приложения была расширена за счет внедрения полноценной ленты сообщений, получаемой из сетевого API, а также механизма локального хранения данных для офлайн-доступа. Главная цель данной части курсовой - обеспечить стабильную работу экрана “Лента” при любом состоянии сети и реализовать единый поток данных, который корректно обновляется как при ручном обновлении, так и при автоматическом получении новых данных.

Основой слоя данных стал репозиторий сообщений. Репозиторий выступает как единая точка доступа к информации для ViewModel и инкапсулирует внутри себя всю логику получения, сохранения и выдачи сообщений. Это позволяет ViewModel не “знать” о том, откуда именно пришли данные - из сети или из базы - и концентрироваться только на подготовке состояния для UI. В репозитории реализованы две ключевые задачи: загрузка сообщений из API и сохранение результата в локальную базу, а также возврат сообщений из локального хранилища в тех ситуациях, когда сеть недоступна или запрос завершился ошибкой.

```
2 Usages
suspend fun refreshMessages(): Result<Unit> {
    return try {
        val likedMap = dao.getLikedMap().associate { it.id to it.liked }

        val dto = api.getMessages()
        val entities = dto.map { it.toEntity() }.map { e ->
            e.copy(liked = likedMap[e.id] ?: false)
        }

        dao.clearAll()
        dao.insertAll( messages = entities)

        Result.success( value = Unit)
    } catch (e: Exception) {
        Result.failure( exception = e)
    }
}
```

Рисунок 5 - MessageRepository с логикой refreshMessages (загрузка из API → сохранение в Room → возврат Result) и методом получения списка из базы (Flow/List)

Сетевой слой реализован на базе Retrofit с поддержкой корутин. Для работы с API был создан интерфейс сервиса, в котором описан HTTP-запрос (например, GET для получения списка сообщений) и формат возвращаемых данных. Retrofit-конфигурация включает базовый URL и конвертер (чаще всего Gson), позволяющий автоматически преобразовывать JSON-ответ в Kotlin-объекты. Ключевой момент этой реализации - использование suspend-функций: сетевой запрос выполняется асинхронно и не блокирует главный поток. Это важно, потому что экран ленты должен оставаться отзывчивым даже при медленном соединении или временной недоступности сервера.

При успешном ответе сети результат преобразуется в модель данных приложения и передается в слой хранения. На этом этапе происходит синхронизация: полученный список сообщений сохраняется в локальную базу Room. Логика сохранения реализована через DAO - объект доступа к данным, который содержит методы вставки, обновления и выборки. Для корректной синхронизации используется стратегия замены или обновления данных: в зависимости от выбранного подхода можно либо полностью обновлять таблицу (удалять старые записи и вставлять новые), либо выполнять upsert-операции. Второй вариант более безопасен, если требуется сохранять локальные пользовательские изменения (например, лайки, которые были добавлены в четвертой лабораторной), но в рамках третьей работы важнее было обеспечить базовый офлайн-кэш и корректную подмену источника данных.

Локальное хранение данных реализовано с использованием Room, включающего три обязательных компонента: Entity, DAO и Database. Entity описывает структуру таблицы сообщений: идентификатор, имя автора, текст сообщения и другие поля, которые приходят с API. DAO предоставляет методы для получения списка сообщений и для сохранения данных. Database является точкой инициализации Room и объединяет все DAO проекта. При этом важно, что чтение сообщений организовано так, чтобы UI мог реагировать на обновления автоматически. Для этого DAO возвращает Flow (или LiveData) - при любом изменении таблицы список обновляется, и подписанный интерфейс мгновенно получает актуальные данные. Таким образом, после успешной сетевой синхронизации достаточно сохранить данные в базу, и экран ленты обновится без дополнительных “ручных” команд.

Для отображения списка сообщений на экране “Лента” был внедрен RecyclerView. Этот компонент оптимален для отображения динамических списков: он переиспользует элементы, экономит память и позволяет

корректно работать даже с большим количеством сообщений. Для RecyclerView создан адаптер и ViewHolder, отвечающие за связывание данных сообщения с элементами интерфейса. Каждый элемент списка отражает ключевые данные сообщения (например, автора и текст). Дополнительно на экране реализована кнопка “Обновить”, которая запускает ручную синхронизацию: ViewModel вызывает метод репозитория refreshMessages, репозиторий выполняет сетевой запрос и сохраняет результат в Room. Такой сценарий позволяет тестировать логику синхронизации и обеспечивает пользователю явный контроль над обновлением ленты.

Отдельно была проработана логика поведения приложения при отсутствии сети. Главный принцип заключается в том, что экран “Лента” не должен становиться пустым или выдавать критическую ошибку только из-за того, что нет интернета. При старте экрана данные загружаются из локальной базы, а затем при наличии сети могут быть обновлены через ручной запрос. Если сеть недоступна, приложение продолжает отображать сообщения, сохраненные ранее. Если база пуста (например, при первом запуске без интернета), приложение отображает корректное состояние пустого списка или информационное сообщение. Таким образом, приложение сохраняет работоспособность в офлайн-режиме и демонстрирует корректное поведение согласно требованиям.

При реализации сетевой загрузки особое внимание уделено обработке ошибок и корректному возврату результата операции. В репозитории результат обновления упаковывается в объект Result: успешная синхронизация возвращается как успех, ошибка сети - как неуспех с возможностью повторной попытки. Это важно для последующей четвертой лабораторной, где эта же логика используется в WorkManager: если сеть недоступна или запрос завершился ошибкой, воркер должен корректно возвращать Result.retry(), а при успехе - Result.success() и показывать уведомление о том, что новые данные получены.

Итогом третьей лабораторной стало построение полноценного контура данных: UI получает список сообщений из ViewModel, ViewModel работает через репозиторий, репозиторий объединяет сетевой API и локальную базу Room, а обновления автоматически отображаются в интерфейсе. Благодаря этому приложение стало устойчивым к изменениям конфигурации, корректно работает при отсутствии сети и обеспечивает единый источник данных для дальнейших функций, добавленных в следующей лабораторной работе.

4 Улучшение интерфейса и фоновая синхронизация

В рамках заключительного этапа разработки приложение было расширено с точки зрения пользовательского интерфейса и системного взаимодействия. Основной целью данной главы стало повышение удобства использования приложения, а также обеспечение автоматической фоновой синхронизации данных и корректной реакции на системные события, такие как изменение состояния сети.

Интерфейс экрана ленты сообщений (рисунок 6) был доработан с использованием компонентов Material Design. Каждый элемент списка сообщений стал представлять собой карточку с аватаром пользователя, именем отправителя и текстом сообщения. Такой подход улучшает визуальное восприятие информации и делает интерфейс более привычным для пользователей мессенджеров. Для отображения списка сообщений по-прежнему используется RecyclerView, однако адаптер был расширен для поддержки более сложной разметки и интерактивных элементов.

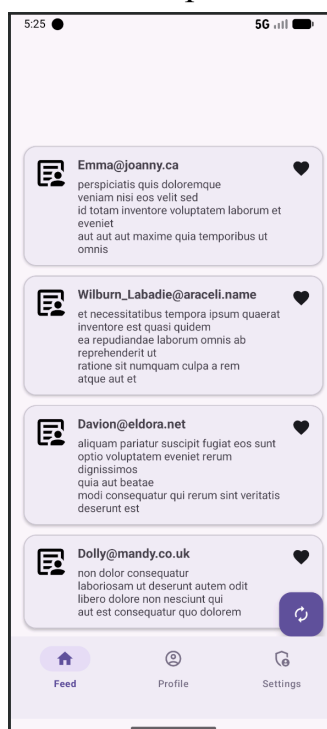


Рисунок 6 - “Лента” с отображением списка сообщений (аватар, имя, текст, лайк)

Дополнительно в интерфейс был интегрирован FloatingActionButton, предназначенный для быстрого обновления списка сообщений вручную. Использование данного элемента соответствует рекомендациям Material Design и делает основное действие на экране интуитивно доступным. Верхняя часть экрана оформлена с применением AppBarLayout, что

обеспечивает корректное поведение панели при прокрутке и визуально отделяет контент от элементов навигации.

В рамках расширения интерактивности была реализована возможность “лайкать” сообщения. При нажатии на соответствующую иконку изменяется визуальное состояние элемента, а информация о лайке сохраняется в локальной базе данных. Это позволяет корректно отображать состояние сообщений даже после перезапуска приложения или при отсутствии сети. Логика обработки кликов реализована во ViewModel, что сохраняет разделение ответственности между UI и бизнес-логикой в соответствии с принципами MVVM.

Ключевым функциональным улучшением стало внедрение фоновой синхронизации данных с использованием WorkManager. Был настроен периодический воркер, который с заданным интервалом выполняет обновление списка сообщений из сети при наличии интернет-соединения. Для воркера заданы ограничения, требующие активного подключения к сети, что предотвращает лишние попытки выполнения задач в офлайн-режиме и снижает нагрузку на систему.

При успешной синхронизации данных приложение отправляет системное уведомление с сообщением о получении новых данных (рисунок 7). Уведомления работают в фоновом режиме и отображаются даже в том случае, если приложение закрыто. Для этого был создан отдельный канал уведомлений, а при запуске приложения реализован запрос соответствующих разрешений, включая разрешение на отправку уведомлений. Логика показа уведомлений инкапсулирована внутри воркера и не зависит от состояния пользовательского интерфейса.

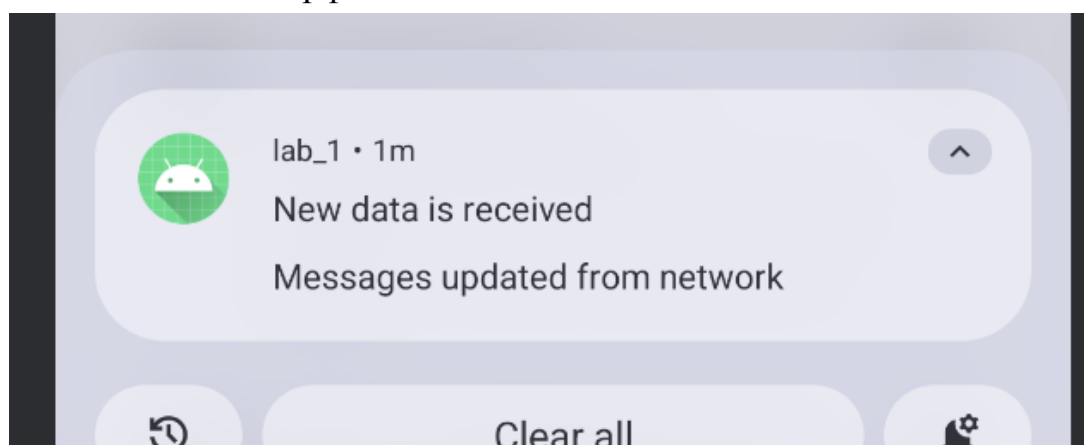


Рисунок 7 - Отображение уведомлений

Приложение также корректно реагирует на изменения состояния сети. При отсутствии интернет-соединения данные загружаются из локальной базы, а синхронизация откладывается до момента восстановления сети.

После появления подключения WorkManager автоматически выполняет отложенную задачу, что обеспечивает целостность данных без участия пользователя. Такой подход демонстрирует использование системных возможностей Android для устойчивой работы приложения в реальных условиях эксплуатации.

В результате выполнения данной главы приложение приобрело законченный пользовательский вид и функциональность, характерную для современных Android-приложений: интерактивный интерфейс, поддержку офлайн-режима, фоновую синхронизацию и работу с уведомлениями. Все изменения были реализованы с сохранением архитектурных принципов MVVM и без нарушения ранее разработанной структуры проекта.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано Android-приложение типа «Мессенджер», последовательно расширяемое в рамках четырех лабораторных работ. Основной целью проекта являлось освоение современных подходов к разработке мобильных приложений под Android, а также формирование навыков проектирования архитектуры, управления состоянием, работы с данными и взаимодействия с системными сервисами операционной системы.

На начальном этапе была реализована базовая структура приложения с использованием одной главной активности и набора фрагментов, объединенных с помощью встроенной навигации Android. Это позволило сформировать устойчивый каркас приложения и обеспечить удобное переключение между экранами. Отслеживание жизненного цикла компонентов дало представление о поведении приложения на различных этапах его работы.

Далее в проекте была внедрена архитектура MVVM, обеспечившая четкое разделение ответственности между пользовательским интерфейсом и бизнес-логикой. Использование ViewModel и реактивных механизмов обновления данных позволило корректно сохранять состояние экранов при изменениях конфигурации и повысило устойчивость приложения к пересозданию компонентов.

Важным этапом разработки стало подключение сетевого взаимодействия и реализация офлайн-доступа. С использованием Retrofit и корутин была настроена загрузка данных из внешнего API, а интеграция базы данных Room обеспечила локальное хранение сообщений и их отображение при отсутствии интернет-соединения. Репозиторий был использован как единая точка доступа к данным, что упростило масштабирование и сопровождение приложения.

На завершающем этапе проект был дополнен улучшенным пользовательским интерфейсом, интерактивными элементами и фоновыми механизмами обновления данных. Использование компонентов Material Design повысило удобство и визуальную целостность интерфейса, а внедрение WorkManager позволило реализовать периодическую синхронизацию данных и отображение системных уведомлений. Приложение научилось корректно реагировать на изменения состояния сети и выполнять обновление данных без участия пользователя.

Полученные в ходе выполнения проекта навыки могут быть использованы при разработке более сложных мобильных приложений и служат основой для дальнейшего изучения платформы Android.